

Offline RL for CPU Scheduling Using Decision Transformers

Objective

- Learn an **offline policy** π that maximizes **expected cumulative return**:

$$\mathbb{E}_{\pi} \left[\sum_{t=1}^T r_t \right]$$

- Constraints:
 - **No online interaction** with the CPU simulator is allowed during learning.

The learned policy must improve upon the suboptimal trajectories.

Environment

- A **simulated CPU system** (single-core or multi-core) produces a stream of jobs over time.
- Each job i has:
 - Arrival time a_i
 - Total CPU burst b_i (unknown to the agent)
 - Optional priority p_i

- The environment generates **state transitions** and rewards based on which job the scheduler chooses to run.
- Rewards reflect **performance metrics**, e.g.:
 - Negative waiting time
 - Negative turnaround time
 - Starvation penalties

Agent

- The **scheduler** selects which process to run at each decision step (or whether to idle).
- The agent observes the **current system state**, which includes:
 - Current queue of waiting jobs
 - Remaining burst times (if partially observable, estimates only)
 - CPU status and recent scheduling history

Trajectories

- A **suboptimal heuristic policy** (e.g., FCFS, Round-Robin, or noisy SJF) generates **logged trajectories** of the form:

$$(s_1, a_1, r_1), (s_2, a_2, r_2), \dots, (s_T, a_T, r_T)$$

- These trajectories are **the only data available** to the agent for learning.

Key Challenges

1. **Delayed reward / long-term credit assignment:**

Early scheduling decisions affect waiting times and starvation much later.

2. **Partial observability:**

Job burst times may be unknown, requiring inference from history.

3. **Distribution shift:**

The agent must improve policy performance without stepping outside the state–action space covered by the logged data.

4. **Sequence dependencies:**

Optimal decisions depend on long histories of prior states and actions.